In Spring, you are not required to read environment variables from the surrounding system explicitly. Spring automatically reads them during the startup phase and adds them to the Spring `Environment` object, making them accessible, just like any other property. For example, if you run a Spring application in an environment where the `MY_ENV_VAR` variable is defined, you can access its value either from the `Environment` interface or using the `@Value` annotation.

On top of that, Spring Boot extends the Spring Framework functionality by allowing you to use environment variables to override Spring properties automatically. For command-line arguments and JVM system properties, you used the same naming convention as the Spring property. However, environment variables have some naming constraints dictated by the operating system. For example, on Linux, the common syntax consists of having all capital letters and words separated by an underscore.

You can turn a Spring property key into an environment variable by making all the letters uppercase and replacing any dot or dash with an underscore. Spring Boot will map it correctly to the internal syntax. For example, a `POLAR_GREETING` environment variable is recognized as the `polar.greeting` property. This feature is called *relaxed binding*.

In the Catalog Service application, you can override the `polar.greeting` property with the following command:

```
$ POLAR_GREETING="Welcome to the catalog from ENV" \
    java -jar build/libs/catalog-service-0.0.1-SNAPSHOT.jar
```

**TIP** On Windows, you can achieve the same result by running `$env:POLAR_GREETING="Welcome to the catalog from ENV"; java -jar build/libs/catalog-service-0.0.1-SNAPSHOT.jar` from a PowerShell console.